Assignment 3

Building Blocks of GAN | Worked solutions to Questions 1-7

1. Transposed convolution output size

Assume dilation 1. Apply the output-size formula to both spatial dimensions:

$$O = (I - 1)S - 2P + K + OP$$

$$O = (8 - 1) \times 2 - 2 \times 1 + 4 + 1 = 17$$

Therefore, the output is **17 x 17**. Output padding contributes the final 1.

2. Increase the stride from 2 to 3

Keeping the Question 1 input and all other parameters fixed:

$$O_{S=3} = (8 - 1) \times 3 - 2 \times 1 + 4 + 1 = 24$$

The output changes from **17 x 17 to 24 x 24**. More generally:

$$\Delta O = (I - 1)(3 - 2) = I - 1 = 7$$

Each spatial dimension increases by 7. The kernel and padding terms remain fixed.

3. General 2D output-size formula

For dilation 1, calculate the height and width separately:

$$H_{out} = (H_{in} - 1)S_H - 2P_H + K_H + OP_H$$

$$W_{out} = (W_{in} - 1)S_W - 2P_W + K_W + OP_W$$

With arbitrary dilation D, the formula for either dimension becomes:

$$O = (I - 1)S - 2P + D(K - 1) + OP + 1$$

4. Upsample 16 x 16 to 32 x 32 without padding

Set padding = 0, output padding = 0, and dilation = 1. One valid choice is **kernel size 2 x 2 and stride 2**:

$$O = (16 - 1) \times 2 + 2 = 32$$

Equivalently, the constraint is 32 = 15S + K, which this choice satisfies.

Normalization and activations

Questions 5-7

5. BatchNorm for [6, 8, 10, 6]

$$\mu_B = \frac{6+8+10+6}{4} = 7.5$$

Use the batch variance with denominator 4 for training-time normalization:

$$\sigma_B^2 = \frac{(-1.5)^2 + (0.5)^2 + (2.5)^2 + (-1.5)^2}{4} = 2.75$$

Without the affine transformation, normalize each value as follows:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

The question does not specify epsilon. Ignoring it for exact unit variance:

$$\hat{x} = \frac{[-3, 1, 5, -3]}{\sqrt{11}} \approx [-0.9045,\ 0.3015,\ 1.5076,\ -0.9045]$$

Check: the exact values sum to zero, and their mean square is (9 + 1 + 25 + 9)/(11 x 4) = 1. A positive epsilon makes the variance slightly below 1.

6. ReLU versus LeakyReLU

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \quad 0 < \alpha < 1$$

Both return x for nonnegative inputs. For negative inputs, ReLU returns 0, whereas LeakyReLU returns alpha times x. Their negative-region slopes are **0 and alpha**, respectively. Here alpha = 0.2, so an input of -5 gives 0 for ReLU and -1 for LeakyReLU.

7. Why prefer LeakyReLU in some deep networks?

ReLU has zero derivative for a negative preactivation. If a unit remains negative for all training examples, it may stop learning (a dying ReLU). LeakyReLU preserves a nonzero derivative, alpha, in that region, allowing gradients to pass and reducing this risk. This can help optimization in a deep network or GAN discriminator. It does not guarantee better performance or remove every source of vanishing gradients.

References

Assignment3.pdf (supplied assignment). PyTorch API documentation:

ConvTranspose2d - official reference

BatchNorm2d - official reference

ReLU - official reference

LeakyReLU - official reference